

Exploring the Google PageRank Algorithm: Mathematical Foundations and Computational Implementation

Dylan Judelson¹ and Caroline Zhuang¹

¹*The University of Texas at Austin
101 E. 21st St
Austin, TX 78712*

ABSTRACT

The PageRank algorithm, originally developed by Larry Page and Sergey Brin at Stanford University, forms the mathematical backbone of modern web search engines. This paper explores the linear algebraic formulation of PageRank, treating the web as a directed graph and representing it through a column-stochastic hyperlink matrix H . We examine the iterative difference equation $x_{k+1} = Hx_k$ that models the evolution of page importance over time, and demonstrate through Theorem 4.9 that this iterative process converges to the dominant eigenvector of H corresponding to eigenvalue $\lambda = 1$. This insight eliminates the need for iteration entirely, reducing the ranking problem to an eigenvalue computation. We apply both methods, iterative and eigenvector-based, to two web graph examples (Figures 4.3 and 4.4) and compare their results. We further discuss conditions required for H to be a valid stochastic matrix, rank the pages in order of importance for both graphs, and reflect on future improvements including the incorporation of randomness via the Google Matrix to address the dangling node problem.

Keywords: keyword 1 (num) — keyword 2 (num) — keyword 3 (num)

INTRODUCTION

The problem of ranking web pages by relevance and importance is central to the functionality of modern search engines. Before algorithmic ranking, web directories were curated manually, a process that quickly became infeasible as the web grew exponentially in the 1990s. The PageRank algorithm, introduced by Brin and Page (Brin & Page 1998), proposed an elegant solution: treat the web as a directed graph, and let the link structure itself determine the relative importance of each page. PageRank has proven powerful well beyond web search. For example, Chen et al. (Chen et al. 2007) applied the algorithm to the *Physical Review* family of journals (1893–2003) and showed that it successfully identifies influential “gems” that are not always apparent from raw citation counts alone.

A page is considered important if many other important pages link to it; a definition that is inherently recursive and amenable to linear algebraic treatment. The core mathematical insight is that web surfing can be modeled as a Markov chain. At each step, a surfer on page i follows one of its outgoing links uniformly at random, transitioning to some page j . The long-run fraction of time the surfer spends on each page defines that page’s rank. This stationary distribution is precisely the dominant eigenvector of the hyperlink matrix H , a column-stochastic matrix encoding the link structure of the web (Langville et al. 2008).

This paper is structured as follows. Section 2 defines the hyperlink matrix and the stochastic matrix condition. Section 3 presents the iterative PageRank process for the graph in Figure 4.3. Section 4 introduces Theorem 4.9 and explains how it simplifies computation. Section 5 applies both methods to Figure 4.3, and Section 6 applies them to Figure 4.4. Section 7 summarizes the algorithm and code. Section 8 discusses future work and reflects on practical PageRank improvement.

THE HYPERLINK MATRIX

Constructing H from Figure 4.3

The web in Figure 4.3 consists of 6 pages with directed links between them. The hyperlink matrix H is constructed so that entry H_{ij} equals $1/(\text{out-degree of page } j)$ if page j links to page i , and 0 otherwise. Each column represents a page, and the column entries represent the equal probability of clicking any one of that page's outgoing links. The resulting matrix is:

$$H = \begin{bmatrix} 0 & 0 & 1/3 & 0 & 0 & 0 \\ 1/2 & 0 & 1/3 & 0 & 0 & 0 \\ 1/2 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/2 & 1 \\ 0 & 0 & 1/3 & 1/2 & 0 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 & 0 \end{bmatrix} \quad (1)$$

Each column sums to 1 and all entries are nonnegative, confirming that H is a valid stochastic matrix.

The Stochastic Matrix Condition

For an $n \times n$ hyperlink matrix H to be a *stochastic matrix*, two conditions must hold: (1) all entries must be nonnegative, and (2) each column must sum to exactly 1.

In the context of the web, this requires that every page has at least one outgoing link. Each column of H corresponds to a page, and its entries represent the probability of following a link from that page to each other page. If a page has no outgoing links (a “dangling node”), its column is entirely zero and cannot sum to 1. Therefore, *every page in the collection must have at least one outgoing hyperlink* for H to be stochastic.

THE ITERATIVE PAGERANK PROCESS

The PageRank vector evolves according to the difference equation:

$$x_{k+1} = Hx_k \quad (2)$$

starting from an initial uniform distribution $x_0 = \frac{1}{n}\mathbf{1}$, where $\mathbf{1}$ is a vector of all ones. Each entry of x_k represents the estimated probability (or rank) of a random web surfer being on that page at step k .

For Figure 4.3 with $n = 6$ pages, we initialize:

$$x_0 = \begin{bmatrix} 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \\ 1/6 \end{bmatrix} \quad (3)$$

and iterate $x_{k+1} = Hx_k$ until convergence. The code below implements this process and plots how each page's rank evolves over 20 iterations:

```
import numpy as np
import matplotlib.pyplot as plt

H = np.matrix([
    [0, 0, 1/3, 0, 0, 0],
    [1/2, 0, 1/3, 0, 0, 0],
    [1/2, 1, 0, 0, 0, 0],
    [0, 0, 0, 0, 1/2, 1],
    [0, 0, 1/3, 1/2, 0, 0],
    [0, 0, 0, 1/2, 1/2, 0]
```

```

73 ])
74
75 n = H.shape[0]
76 x = np.matrix([[1/6]]*n)
77 iterations = 20
78 rank_history = np.zeros((iterations + 1, n))
79 rank_history[0] = x.flatten()
80
81 for i in range(iterations):
82     x = H * x
83     rank_history[i+1] = x.flatten()
84
85 plt.figure(figsize=(10, 6))
86 for page in range(n):
87     plt.plot(range(iterations + 1), rank_history[:, page],
88             marker='o', label=f'Page {page+1}')
89 plt.xlabel('Iteration')
90 plt.ylabel('PageRank')
91 plt.title('PageRank Evolution Over Iterations (Figure 4.3)')
92 plt.legend()
93 plt.grid(True)
94 plt.show()

```

95 The plot shows that the ranks converge quickly, within roughly 10 iterations, with pages 1, 2, and 3 converging
96 toward low values near zero, while pages 4, 5, and 6 stabilize at higher values, reflecting the link structure of the
97 graph.

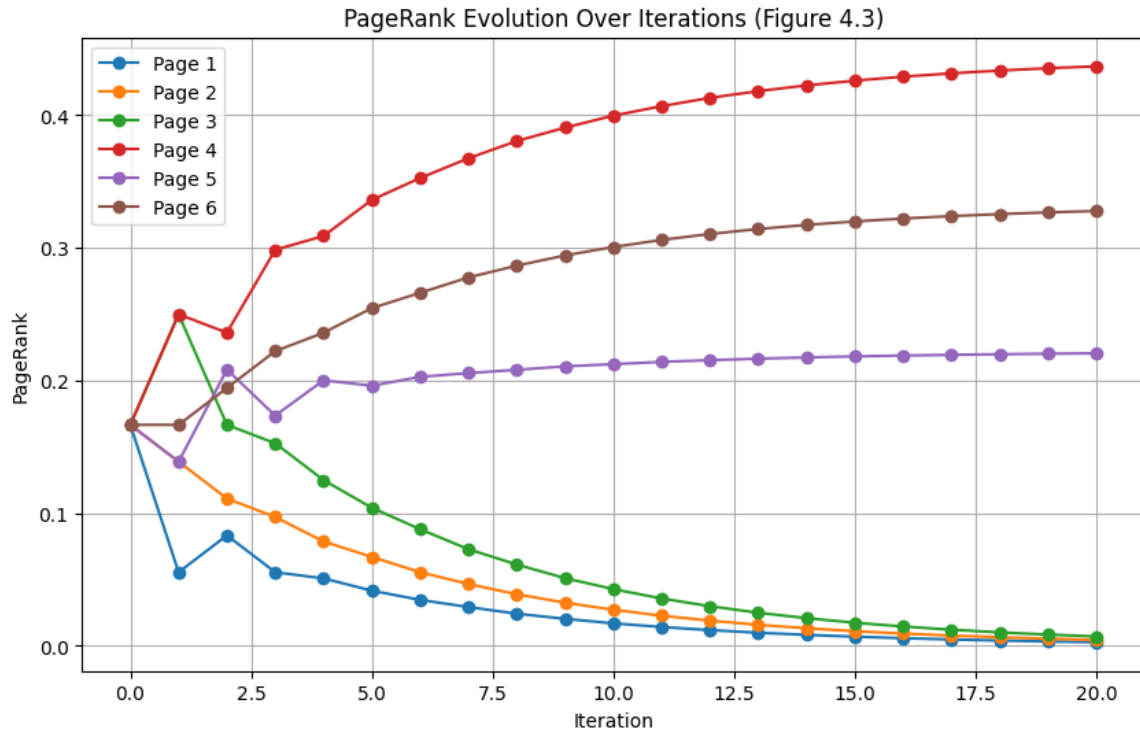


Figure 1. PageRank evolution over 20 iterations for the web graph in Figure 4.3. Pages 1–3 converge near zero while pages 4–6 stabilize at higher rank values.

THEOREM 4.9 AND THE DOMINANT EIGENVECTOR

Statement of Theorem 4.9

Theorem 4.9. If A is an $n \times n$ stochastic matrix and x_0 is some initial vector for the difference equation $x_{n+1} = Ax_n$, then the steady state vector is:

$$x_{\text{equil}} = \lim_{k \rightarrow \infty} A^k x_0 = v \quad (4)$$

where v is the unique eigenvector corresponding to the eigenvalue $\lambda = 1$ that is normalized so that its entries sum to 1 (i.e., v is a probability vector).

A stochastic matrix always has $\lambda = 1$ as an eigenvalue. From the given theorem, all other eigenvalues λ_j satisfy $|\lambda_j| < 1$, which implies that $\lambda_j^k \rightarrow 0$ as $k \rightarrow \infty$. Therefore, when x_0 is expressed in the eigenbasis of A , all components except the one along the $\lambda = 1$ eigenvector vanish in the limit. Since we start with a probability vector x_0 (nonnegative entries summing to 1) and A preserves this property (being stochastic), the limiting vector v is also a nonnegative probability vector with entries summing to 1.

Implication for PageRank

Instead of having to perform the iterative process, we only need to find the eigenvector that corresponds to the eigenvalue of 1, normalized so that its entries sum to 1, as seen from the theorem above. This turns the problem into a linear algebra one instead of a long, tedious iterative process.

PAGERANK RESULTS: FIGURE 4.3

The hyperlink matrix H for the 6-page web in Figure 4.3 was given in Section 2. Starting from the uniform initial vector $x_0 = (1/6)\mathbf{1}$, both the iterative method (20 iterations) and the dominant eigenvector method (Theorem 4.9) were applied.

The resulting PageRank vector is:

$$\mathbf{x} \approx \begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.0000 \\ 0.4444 \\ 0.2222 \\ 0.3333 \end{bmatrix}. \quad (5)$$

Page ranking (highest to lowest importance): **Page 4** > **Page 6** > **Page 5** > **Page 3** > **Page 2** > **Page 1**. Pages 1–3 have essentially zero long-term importance because they feed traffic into the strongly connected component of pages 4–6 but receive little return traffic.

The convergence plot (Figure 1) shows rapid stabilization, confirming the effectiveness of the iterative process and the exactness of the eigenvector approach.

PAGERANK RESULTS: FIGURE 4.4

Hyperlink Matrix and Initial State

The web in Figure 4.4 consists of 8 pages. The hyperlink matrix is:

$$H = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1/3 & 0 \\ 1/3 & 0 & 1/3 & 0 & 0 & 0 & 0 & 0 \\ 1/3 & 1 & 0 & 0 & 1/4 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 1/4 & 1 & 0 & 1 \\ 0 & 0 & 1/3 & 1/3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 1/4 & 0 & 0 & 0 \\ 1/3 & 0 & 1/3 & 0 & 1/4 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/3 & 0 & 0 & 1/3 & 0 \end{bmatrix} \quad (6)$$

The initial state vector, assuming equal probability across all 8 pages, is:

$$x_0 = \begin{bmatrix} 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \\ 1/8 \end{bmatrix} \quad (7)$$

Method 1: Iterative Difference Equation

```

132 H_2 = np.matrix([
133     [0, 0, 0, 0, 0, 0, 1/3, 0],
134     [1/3, 0, 1/3, 0, 0, 0, 0, 0],
135     [1/3, 1, 0, 0, 1/4, 0, 1/3, 0],
136     [0, 0, 0, 0, 1/4, 1, 0, 1],
137     [0, 0, 1/3, 1/3, 0, 0, 0, 0],
138     [0, 0, 0, 1/3, 1/4, 0, 0, 0],
139     [1/3, 0, 1/3, 0, 1/4, 0, 0, 0],
140     [0, 0, 0, 1/3, 0, 0, 1/3, 0]
141 ])
142 n = H_2.shape[0]
143 x_2 = np.matrix([[1/8]]*n)
144 for i in range(20):
145     x_2 = H_2 * x_2
146 print(x_2)

```

Result after 20 iterations:

$$x_{\text{iter}} \approx \begin{bmatrix} 0.02913 \\ 0.05193 \\ 0.12584 \\ 0.29860 \\ 0.14243 \\ 0.13551 \\ 0.08723 \\ 0.12933 \end{bmatrix} \quad (8)$$

Method 2: Dominant Eigenvector (Theorem 4.9)

```

150 w_2, v_2 = np.linalg.eig(H_2)
151 x_eig2 = np.real(v_2[:, 0])
152 x_eig2 = x_eig2 / np.sum(x_eig2)
153 print(x_eig2)

```

Result:

$$x_{\text{eig}} \approx \begin{bmatrix} 0.02903 \\ 0.05161 \\ 0.12581 \\ 0.30000 \\ 0.14194 \\ 0.13548 \\ 0.08710 \\ 0.12903 \end{bmatrix} \quad (9)$$

Comparison and Page Ranking

The two methods produce nearly identical results. The page importance ranking (highest to lowest) is: **Page 4** > **Page 5** > **Page 6** > **Page 8** > **Page 3** > **Page 7** > **Page 2** > **Page 1**. Page 4 dominates with roughly 30% of the rank, reflecting that it receives links from multiple important pages (pages 6 and 8 both link exclusively to it). The small numerical discrepancies between the two approaches arise because the iterative method uses only 20 steps rather than the true limit; the eigenvector method gives the exact steady-state result.

ALGORITHM AND CODE SUMMARY

The PageRank computation in this paper follows two parallel approaches, both implemented in Python using `numpy`.
Method 1 — Iterative Difference Equation:

1. Construct the $n \times n$ hyperlink matrix H from the directed web graph, where $H_{ij} = 1/(\text{out-degree of } j)$ if $j \rightarrow i$, else 0.
2. Initialize $x_0 = \frac{1}{n} \mathbf{1}$ (uniform distribution).
3. Repeatedly compute $x_{k+1} = Hx_k$ until convergence (or a fixed number of iterations).
4. The resulting vector x is the approximate PageRank.

Method 2 — Dominant Eigenvector (Theorem 4.9):

1. Construct H as above.
2. Compute the eigendecomposition of H using `np.linalg.eig(H)`.
3. Identify the eigenvector corresponding to $\lambda = 1$ (the first eigenvalue in practice).
4. Normalize this eigenvector so its entries sum to 1.
5. The normalized vector is the exact steady-state PageRank vector.

Method 2 is strictly preferred for exact results. Method 1 is useful for visualizing convergence behavior, as shown in the iteration plot for Figure 4.3. Both methods scale to larger graphs, though for very large-scale web graphs (billions of pages), more efficient sparse matrix techniques and power iteration methods are typically employed in practice.

CONCLUSION AND FUTURE WORK

Improving Your Own Website's PageRank

The PageRank results demonstrate that a page's importance is largely determined by *who links to it*, not just how many pages link to it. To improve the PageRank of a personal or project website, the most effective strategies are: (1) obtaining links from high-PageRank pages (e.g., university department pages, published project repositories, or established blogs); (2) ensuring that any pages you control link back to your main page to consolidate rank; and (3) minimizing dangling nodes on your site, since pages with no outgoing links do not pass rank forward and can degrade the stochastic structure of the local subgraph.

Future Work & Adding Randomness: The Google Matrix

In the basic model studied in this paper, the hyperlink matrix H assumes that a random surfer always follows an outgoing link. Real web graphs, however, frequently contain “dangling nodes” (pages with no outgoing links), which break the stochastic property or cause rank to leak out of the system.

A standard way to address this limitation and make the model more realistic is to introduce a small probability of “teleportation.” With probability α (typically $\alpha = 0.85$), the surfer follows a link chosen uniformly from the current page's outgoing links. With probability $1 - \alpha$, the surfer jumps to any page in the web chosen uniformly at random.

This modification produces the **Google matrix**:

$$G = \alpha H + (1 - \alpha) \frac{1}{n} \mathbf{1} \mathbf{1}^T, \quad (10)$$

where $\mathbf{1}$ is the all-ones vector and n is the number of pages. The matrix G is always positive and irreducible (hence primitive for $\alpha \in (0, 1)$), guaranteeing a unique positive steady-state distribution regardless of dangling nodes.

The parameter $\alpha = 0.85$ is the conventional choice in the original Google implementation, balancing the influence of the actual link structure against occasional random jumps. This formulation represents a natural and important generalization of the pure hyperlink model explored in this project.

The damping/teleportation approach described above is analyzed in detail by (Gleich 2015), who shows how the Google matrix formulation extends PageRank far beyond the basic hyperlink model.

Acknowledgements

The author thanks the CS323E Teaching Assistants for their helpful discussions during the course of this project.

REFERENCES

- | | |
|--|---|
| <p>Brin, S., & Page, L. 1998, Computer Networks and ISDN Systems, 30, 107, doi: 10.1016/S0169-7552(98)00110-X</p> <p>Chen, P., Xie, H., Maslov, S., & Redner, S. 2007, Journal of Informetrics, 1, 8, doi: 10.1016/j.joi.2006.06.001</p> | <p>Gleich, D. F. 2015, SIAM Review, 57, 321, doi: 10.1137/140976649</p> <p>Langville, A., Meyer, C., & Fernández, P. 2008, The Mathematical Intelligencer, 30, 68, doi: 10.1007/BF02985759</p> |
|--|---|